

个人答辩报告

12311153 廖子韬

SUSTC

May 14, 2025

Content

① 负责工作简述

② IFetch

③ DMem

④ InputModule

负责工作简述

IFetch
000

DMem
000

InputModule
0000

负责工作简述

- ① IFetch: CPU 指令获取单元
- ② DMem: CPU (数据) 内存单元
- ③ InputModule: CPU 输入模块
- ④ 基本测试场景

负责工作简述
0

IFetch
●00

DMem
000

InputModule
0000

IFetch-Design Source

```
1 module IFetch(  
2     input clk,  
3     input rst,  
4     input [31:0] imm32,  
5     input branch,  
6     input zero,  
7     input is_jal,  
8     input is_jalr,  
9     input [31:0] new_pc,  
10    input en_pc,  
11    output [31:0] inst,  
12    output reg [31:0] out_pc  
13);  
14    reg [31:0] pc;  
15    wire [13:0] addr;  
16    prgrom urom(.clka(clk), .addra(addr),  
17                .douta(inst));  
18    always @(posedge clk) begin  
19        out_pc<=pc;  
20    end  
21    parameter base_address = 32'  
22        h0000_3000;  
23    reg[31:0] next_pc;  
24    assign addr = (pc[13:0]-  
25        base_address) >> 2;
```

```
1     always @(*) begin  
2         if (!en_pc) begin  
3             next_pc=pc;  
4         end else if (is_jalr) begin  
5             next_pc = new_pc;  
6         end else if (branch && zero ||  
7             is_jal) begin  
8             next_pc = pc + imm32;  
9         end else begin  
10            next_pc = pc + 32'd4;  
11        end  
12    end  
13    always @(negedge clk, negedge rst)  
14    begin  
15        if(~rst) begin  
16            pc <= base_address;  
17        end else begin  
18            pc <= next_pc;  
19        end  
20    end  
21 endmodule
```

IFetch-Testbench

```

1 initial clk = 0;
2 always #10 clk = ~clk;
3 initial begin
4   rst = 0;
5   en_pc = 1;
6   branch = 0;
7   zero = 0;
8   is_jal = 0;
9   is_jalr = 0;
10  imm32 = 32'd8;
11  new_pc = 32'h0000_4000;
12  #25;
13  rst = 1;
14  // normal increment: PC=0x3000,
15  // next=0x3004
16  #20;
17  // branch taken: PC += imm32 (8)
18  branch = 1; zero = 1; imm32 =
19  32'd16;

```

```

1 // jal: PC += imm32
2 is_jal = 1; imm32 = 32'd32;
3 #20;
4 is_jal = 0;
5
6 // freeze PC: en_pc=0
7 en_pc = 0; imm32 = 32'd100;
8 #20;
9 en_pc = 1;
10
11 // jalr: PC = new_pc
12 is_jalr = 1; new_pc = 32'
13   h0000_5000;
14 #20;
15 is_jalr = 0;
16 // finish
17 #20;
18 $finish;
19 end

```

Figure: Testbench of IFetch

IFetch-Waveform

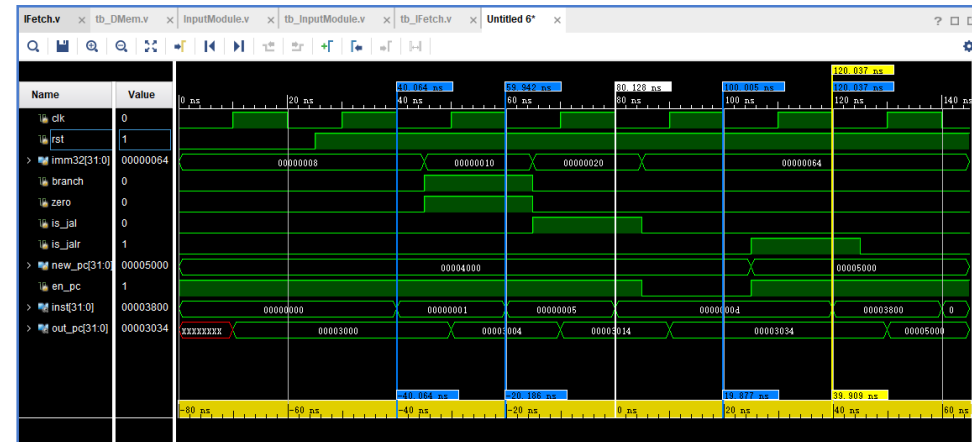


Figure: Waveform of IFetch

DMem-Design source

```

1 module DMem(
2   input clk,
3   input mem_read,
4   input mem_write,
5   input en_pc,
6   input [31:0] addr,
7   input [31:0] din,
8   input [2:0] funct3,
9   output [31:0] dout
10);
11 parameter SB=3'b000,SH=3'b001,SW
12   =3'b010;
13 reg [3:0] write_byte;
14 reg [31:0] tmp_write_data;
15 program udram(.clka(~clk),
16   .wea(en_pc ? write_byte : 0),
17   .addra(addr[15:2]),
18   .dina(tmp_write_data),
19   .douta(dout));

```

```

1 always@(*) begin
2   if (mem_write) begin
3     case(funct3)
4       SW: write_byte = 4'b1111;
5       tmp_write_data = din;
6       SH: tmp_write_data = {2{din
7         [15:0]}};
8       case(addr[1])
9         0: write_byte = 4'b0011;
10        1: write_byte = 4'b1100;
11        default: write_byte = 4'b0;
12      endcase
13      SB: tmp_write_data = {4{din[7:0]}};
14      case(addr[1:0])
15        0: write_byte = 4'b0001;
16        1: write_byte = 4'b0010;
17        2: write_byte = 4'b0100;
18        3: write_byte = 4'b1000;
19        default: write_byte = 4'b0;
20      endcase
21      default: write_byte = 4'b0000;
22    endcase
23  end else write_byte = 4'b0;
24 end endmodule

```

DMem-Testbench

```

1 initial clk = 0;
2 always #10 clk = ~clk;
3 initial begin
4   // Test SW: write full word
5   en_pc = 1;
6   mem_write = 1;
7   funct3 = 3'b010; // SW
8   addr = 32'h0000_0004;
9   din = 32'hDEAD_BEEF;
10  #20;
11  mem_write = 0;
12  mem_read = 1;
13  #20; // Test SH lower half
14  mem_read = 0;
15  mem_write = 1;
16  funct3 = 3'b001; // SH
17  addr = 32'h0000_0008;
18  din = 32'h0000_1234;
19  #20; mem_write = 0;
20  mem_read = 1;

```

```

1 #20; // Test SH upper half
2 mem_read = 0;
3 mem_write = 1;
4 addr = 32'h0000_000A;
5 #20; mem_write = 0;
6 mem_read = 1;
7 #20; // Test SB byte 0
8 mem_read = 0;
9 mem_write = 1;
10 funct3 = 3'b000; // SB
11 addr = 32'h0000_000C;
12 din = 32'h0000_00AA;
13 #20; mem_write = 0;
14 mem_read = 1;
15 #20; // Test SB byte 3
16 mem_read = 0;
17 mem_write = 1;
18 addr = 32'h0000_000F;
19 #20; mem_write = 0;
20 mem_read = 1;
21 #20; $finish;
22 end

```

Figure: DMem-Testbench

DMem-Waveform

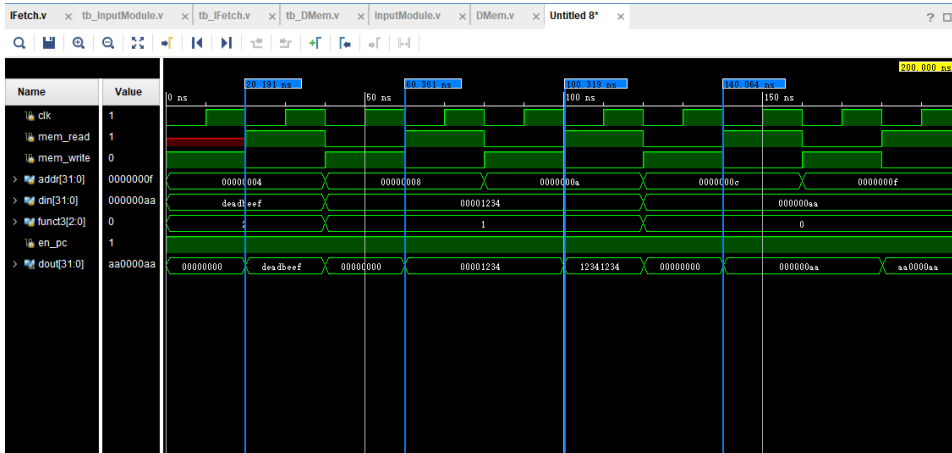


Figure: Waveform of DMem

InputModule-Design Source

```

1 module InputModule(
2     input clk,
3     input clk_de,
4     input rst,
5     input [7:0] sw_input,
6     input done,
7     output [31:0] input_data,
8     output reg done_input);
9     assign input_data = {{24{sw_input
10         [7]}},sw_input};
11     reg done_stable;
12     reg [3:0] done_counter;
13     reg next_done_input, period_done,
14         next_period_done;
15     always @(posedge clk_de , negedge
16         rst) begin
17         ... Debounce of the button
18     end
19 endmodule

```

```

1 always @(posedge clk, negedge rst)
2     if (~rst)
3         done_input <= 0;
4         period_done <= 0;
5     else
6         done_input <= next_done_input;
7         period_done <= next_period_done;
8 always @(*)
9     if (done_stable)
10        if (done_input)
11            next_period_done = 1;
12            next_done_input = 0;
13        else
14            next_period_done =
15                period_done;
16            if (~period_done)
17                next_done_input = 1;
18            else next_done_input = 0;
19        else
20            next_period_done = 0;
21            next_done_input = 0;
22 endmodule

```

Figure: Code of InputModule

InputModule-Testbench

```

1  initial begin
2  // apply reset
3  rst = 0; done = 0; sw_input = 8'h0;
4  #20; rst = 1;
5  // test sign-extension of sw_input
6  sw_input = 8'hF0; #10;
7  if (input_data != 32'hFFFF_FFF0)
8      $fatal("Sign-ext_ufailed");
9
10 sw_input = 8'h7A; #10;
11 if (input_data != 32'h0000_007A)
12     $fatal("Zero-ext_ufail");
13
14 // assert done for one clk_de cycle
15 //   + stable since threshold=1
16 done = 1; #1000;
17 done = 0; #1000;
18 if (done_input != 1) $fatal("
19     Debounce_fail:ufirst_pulse");

```

```

1 // further pulses ignored until
  cleared
2 done = 1; #1000;
3 done = 0; #1000;
4 if (done_input != 0) $fatal("
    Debounce_fail: should_clear_
    after_done");
5
6 // second valid pulse after release
7 done = 1; #1000;
8 done = 0; #1000;
9 if (done_input != 1)
    $fatal("Debounce_fail: second_
    pulse");
10 #100; $finish;
11
12 end

```

Figure: Code of InputModule

InputModule-Waveform

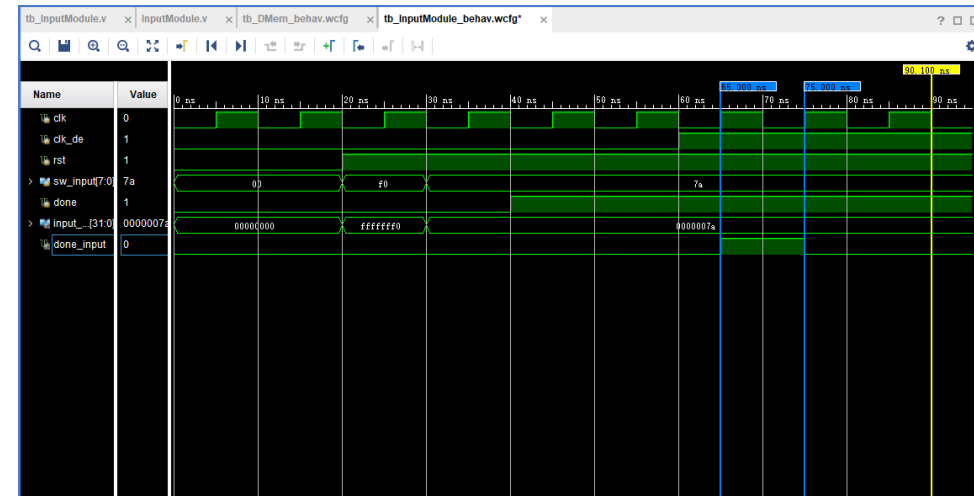


Figure: Waveform of InputModule